Alexandria University

Faculty of Engineering

Computer and Systems Engineering Department

Assignment 2

Heart Failure Classification Problem

Course: CSE: Pattern Recognition

Team Members

Name	Student ID
Omar Khaled Hussien	22010962
Yaseen Asaad Ahmed Farid	22011349
Ahmed Mohamed Mansour	22010576

Supervised by:

Prof. Dr. Marwan Torki & **Eng. Nour Eddine**

Contents

1 Introduction **3**
1.1 Problem Statement . . . 3
1.2 Assignment Objectives . . . 3
1.3 Tools and Libraries . . . 3

2 Dataset and Data Preparation **4**
2.1 Dataset Overview . . . 4
2.2 Class Distribution . . . 4
2.3 Categorical Feature Encoding . . . 5
2.4 Train / Validation / Test Split . . . 5
2.5 Feature Standardization . . . 6

3 Model Implementations and Results **7**
3.1 Model 1 — Decision Tree (From Scratch) . . . 7
3.1.1 Theoretical Background . . . 7
3.1.2 Implementation Details . . . 7
3.1.3 Hyperparameter Tuning . . . 7
3.1.4 Test Set Results . . . 8
3.2 Model 2 — Bagging Ensemble (From Scratch) . . . 9
3.2.1 Theoretical Background . . . 9
3.2.2 Implementation Details . . . 9
3.2.3 Hyperparameter Tuning . . . 9
3.2.4 Test Set Results . . . 10
3.3 Model 3 — AdaBoost Ensemble (From Scratch) . . . 11
3.3.1 Theoretical Background . . . 11
3.3.2 Implementation Details . . . 12
3.3.3 Hyperparameter Tuning . . . 12
3.3.4 Test Set Results . . . 13
3.4 Bonus Model — Random Forest (From Scratch) . . . 14
3.4.1 Theoretical Background . . . 14
3.4.2 Implementation Details . . . 14
3.4.3 Hyperparameter Tuning . . . 15
3.4.4 Test Set Results . . . 16

4 Evaluation and Results **18**
4.1 Performance Summary . . . 18
4.2 Per-Class Metrics . . . 18
4.3 Visual Comparison . . . 19
4.4 Side-by-Side Confusion Matrices . . . 19
4.5 Most confused class analysis . . . 20

5 Analysis and Comparison **20**
5.1 Which Model Performed Best and Why? . . . 20
5.2 Decision Tree vs. Ensemble Methods . . . 20
5.3 Misclassification Pattern Analysis . . . 21
5.3.1 False Negatives (FN) — Missed Heart Disease Cases . . . 21
5.3.2 False Positives (FP) — Healthy Patients Flagged as High-Risk . . . 21

5.3.3 Error Type Trade-offs Across Models 22
5.4 Summary of Insights 22

1 Introduction

Cardiovascular diseases (CVDs) are the leading cause of death globally, claiming an estimated 17.9 million lives each year. Heart failure is one of the most severe manifestations of CVDs, arising when the heart muscle cannot pump enough blood to meet the body's demands. Early, data-driven prediction of heart failure risk can enable timely medical intervention, potentially saving lives and reducing healthcare costs.

Machine learning provides a powerful toolkit for this task: by learning patterns from clinical measurements, classifiers can flag high-risk patients automatically and assist clinicians in prioritizing care. This assignment explores three classical tree-based classification algorithms — a single Decision Tree, a Bagging Ensemble, and an AdaBoost Ensemble — all implemented **from scratch** in Python, and evaluates them on the publicly available Heart Failure Prediction dataset. As a bonus, a **Random Forest** classifier was also implemented from scratch and is reported separately.

1.1 Problem Statement

Given a set of 11 clinical features for a patient (age, blood pressure, cholesterol, etc.), predict the **binary outcome**: `HeartDisease = 0` (no heart disease) or `HeartDisease = 1` (heart disease present). The dataset contains 918 patient records sourced from five international medical institutions and made available on Kaggle.

1.2 Assignment Objectives

- Implement a **Decision Tree** classifier from scratch using the Information Gain splitting criterion.
- Implement a **Bagging Ensemble** from scratch using bootstrap aggregating over multiple decision trees.
- Implement an **AdaBoost Ensemble** from scratch using sequential, weighted boosting.
- *[Bonus]* Implement a **Random Forest** from scratch using bootstrap aggregating with random feature subsampling.
- Tune hyperparameters on the validation set and perform final evaluation on the held-out test set.
- Compare all models using Accuracy, F1-Score, and Confusion Matrices and provide detailed analysis.

1.3 Tools and Libraries

All code was written in Python 3 inside a Jupyter Notebook:

- **NumPy** and **Pandas** — array operations, data loading, and manipulation.
- **Matplotlib** and **Seaborn** — all plots and visualizations.
- **Scikit-learn** — *only* for `train_test_split`, `StandardScaler`, and metric functions (`accuracy_score`, `f1_score`, `confusion_matrix`). No sklearn classifier was used.

A fixed random seed of **42** was applied to every random operation to ensure complete reproducibility.

2 Dataset and Data Preparation

2.1 Dataset Overview

The Heart Failure Prediction dataset was compiled by combining five heart-disease datasets from Cleveland, Hungary, Switzerland, Long Beach VA, and Stalog, resulting in **918 unique patient records**. Each record contains 11 clinical features and one binary target label.

Table 1: Complete feature descriptions for the Heart Failure Prediction dataset

Feature	Type	Description and Range
`Age`	Numerical	Age of the patient in years (28–77)
`Sex`	Categorical	Biological sex: M (Male), F (Female)
`ChestPainType`	Categorical	TA = Typical Angina; ATA = Atypical Angina; NAP = Non-Anginal Pain; ASY = Asymptomatic
`RestingBP`	Numerical	Resting blood pressure in mm Hg (0–200)
`Cholesterol`	Numerical	Serum cholesterol in mg/dL (0–603)
`FastingBS`	Binary	1 if fasting blood sugar > 120 mg/dL, else 0
`RestingECG`	Categorical	Normal, ST (ST-T wave abnormality), LVH (left ventricular hypertrophy)
`MaxHR`	Numerical	Maximum heart rate achieved (60–202)
`ExerciseAngina`	Categorical	Exercise-induced angina: Y (Yes), N (No)
`Oldpeak`	Numerical	ST depression induced by exercise (−2.6 to 6.2)
`ST_Slope`	Categorical	Slope of peak exercise ST segment: Up, Flat, Down
`HeartDisease`	**Target**	**0 = No Heart Disease, 1 = Heart Disease**

A scan for missing values confirmed **zero missing entries** across all 12 columns. The dataset is complete and required no imputation.

2.2 Class Distribution

The dataset contains **508 positive samples (55.3%)** and **410 negative samples (44.7%)**. This mild class imbalance is important context when interpreting F1-scores, and motivated the use of stratified splitting.

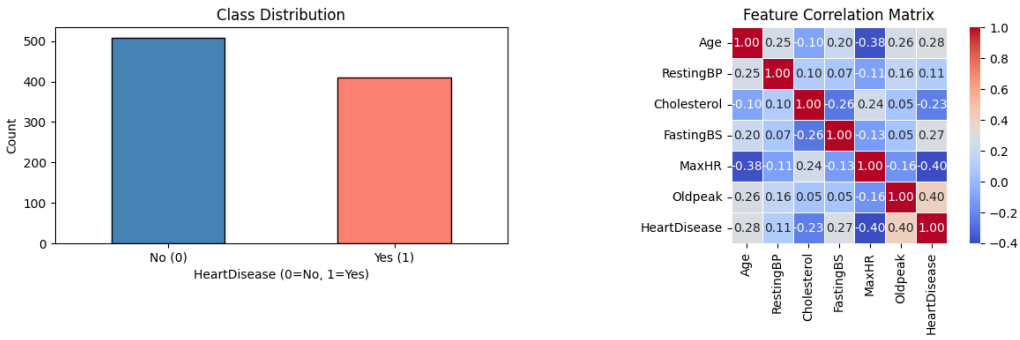

Figure 1: Left: bar chart of class counts (508 heart disease vs. 410 no disease). Right: distribution of key numerical features by class. Patients with heart disease tend to have lower maximum heart rate and higher ST depression (Oldpeak).

2.3 Categorical Feature Encoding

Five features are categorical and must be converted to numeric form. We applied **one-hot encoding** with `drop_first=False` (retaining all categories) to avoid arbitrary ordinal assignments:

```
Shape before encoding:  (918, 12)  | 11 features + 1 target
Shape after encoding:   (918, 21)  | 20 features + 1 target
```

The 14 new binary columns : `Sex_F/M`, `ChestPainType_ASY/ATA/NAP/TA`, `RestingECG_LVH/Normal/ST`, `ExerciseAngina_N/Y`, `ST_Slope_Down/Flat/Up`.

2.4 Train / Validation / Test Split

The data was divided using **stratified sampling** with `random_state=42`, performed in two steps: first 20% test, then splitting the remaining 80% into 70% train / 10% validation.

Table 2: Dataset split summary

Split	Samples	% Total	No Disease (0)	Heart Disease (1)	Purpose
Training	642	69.9%	287 (44.7%)	355 (55.3%)	Model fitting
Validation	92	10.0%	41 (44.6%)	51 (55.4%)	Hyperparameter tuning
Test	184	20.0%	82 (44.6%)	102 (55.4%)	Final evaluation
Total	**918**	100%	410 (44.7%)	508 (55.3%)	—

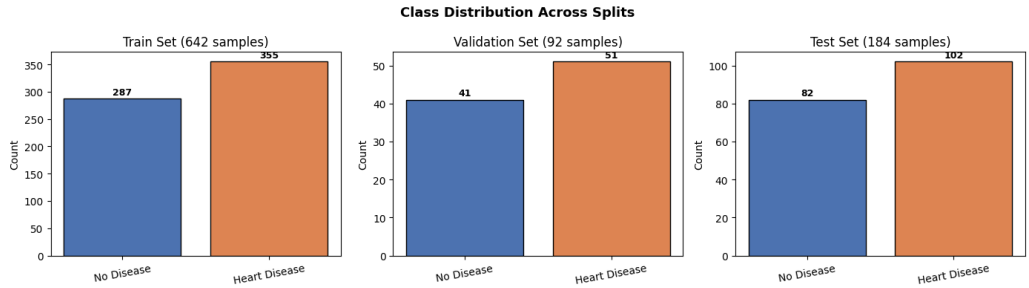

Figure 2: Class balance across all three splits. Stratified sampling ensures the 55.3%/44.7% ratio is preserved in every subset, preventing any split from being artificially easier or harder.

The test set was strictly locked away: never used during model selection or hyperparameter tuning.

2.5 Feature Standardization

The six numerical features were standardized using `StandardScaler` fitted *only* on the training set, then applied to validation and test sets to prevent data leakage:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

One-hot encoded binary columns were left unchanged (already in $\{0, 1\}$).

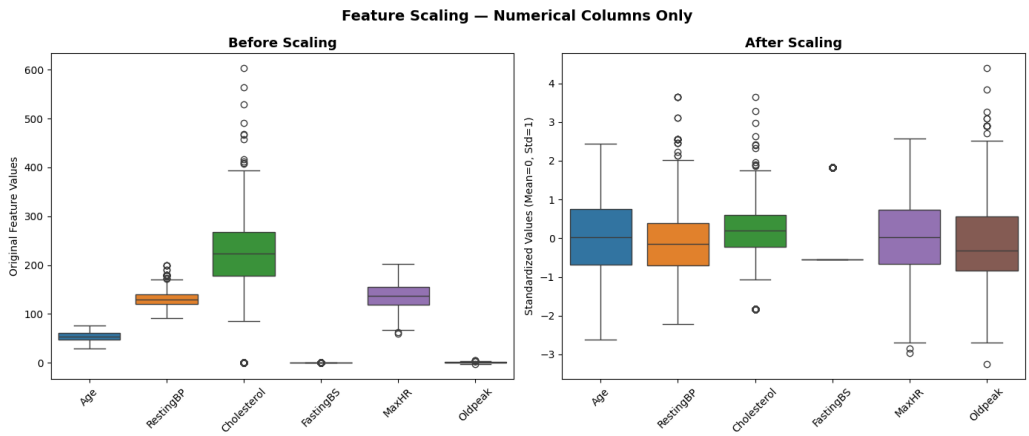

Figure 3: Numerical feature distributions before (left) and after (right) standardization. Post-scaling: mean ≈ 0, std ≈ 1 for all numerical features.

3 Model Implementations and Results

All three required classifiers (plus a bonus Random Forest) were implemented entirely from scratch in Python without using any sklearn classifier. Each model was: (1) trained on the training set, (2) tuned on the validation set, (3) evaluated on the test set.

3.1 Model 1 — Decision Tree (From Scratch)

3.1.1 Theoretical Background

A Decision Tree partitions the feature space by recursively selecting splits that maximize class separability. Each internal node asks "Is feature $j \leq t$?" and routes samples left ($\leq t$) or right ($> t$). Each leaf stores the majority class of samples that reach it.

The splitting criterion is **Information Gain (IG)**, based on Shannon Entropy:

$$H(S) = -\sum_{c \in \{0,1\}} p_c \log_2(p_c)$$

$$\mathrm{IG}(S,\, j,\, t) = H(S) - \frac{|S_L|}{|S|} H(S_L) - \frac{|S_R|}{|S|} H(S_R)$$

where $S_L = \{x \in S : x_j \leq t\}$ and $S_R = \{x \in S : x_j > t\}$. At each node, we evaluate all features and all midpoint thresholds between consecutive unique values to find (j^*, t^*) maximizing IG.

3.1.2 Implementation Details

Decision Tree — Key Implementation Choices

- **Node structure:** Each node stores `feature_idx`, `threshold`, `left/right` children, and `value` (leaf-only).
- **Thresholds:** Midpoints between consecutive sorted unique feature values — covers all binary splits efficiently.
- **Leaf prediction:** Majority class of training samples at that node.
- **Stopping criteria:** Max depth reached **or** fewer than `min_samples_split` samples **or** fewer than `min_samples_leaf` per child **or** pure node (entropy = 0).
- **Recursion:** Depth-first growth; no post-hoc pruning.

3.1.3 Hyperparameter Tuning

A grid search over **120 combinations** evaluated on the validation set:

Parameter	Values Searched
`max_depth`	3, 5, 7, 10, 15, None
`min_samples_split`	2, 5, 10, 20
`min_samples_leaf`	1, 3, 5, 10, 15

Results consistently clustered around `max_depth=5`, confirming that deeper trees overfit on this 642-sample training set.

Table 3: Best Decision Tree hyperparameters (validation accuracy = 85.87%)

Parameter	Best Value	Rationale
`max_depth`	5	Prevents overfitting; deeper trees degraded validation accuracy
`min_samples_split`	2	Allows splitting as long as leaf size constraint is met
`min_samples_leaf`	10	Ensures each leaf is backed by sufficient samples

3.1.4 Test Set Results

Decision Tree (Scratch) — Test Set Performance

Accuracy: 84.24% **F1-Score: 0.8585**

```
Classification Report:
                   precision    recall  f1-score   support
 No Heart Disease      0.83      0.82      0.82        82
    Heart Disease      0.85      0.86      0.86       102
         accuracy                         0.84       184
        macro avg      0.84      0.84      0.84       184
     weighted avg      0.84      0.84      0.84       184
```

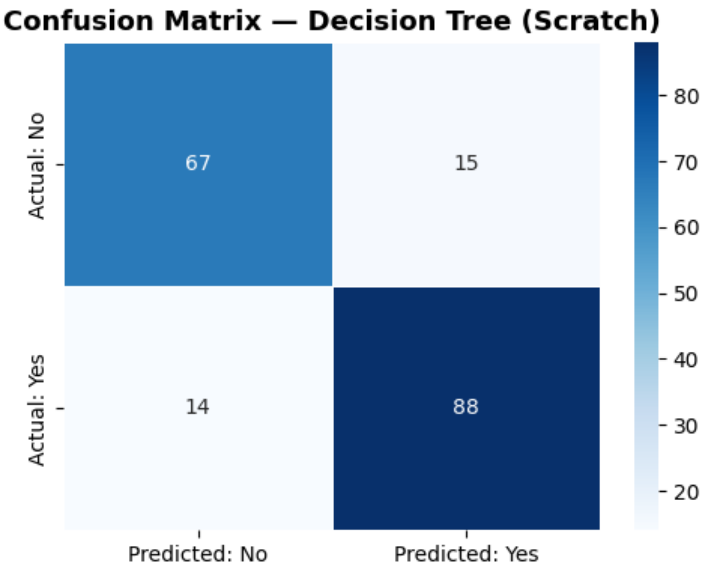

Figure 4: Confusion matrix — Decision Tree. TN=67, FP=15, FN=14, TP=88. Total errors: 29 out of 184 samples (15.76% error rate).

Analysis: The tree achieves balanced precision and recall. Higher recall for Heart Disease (86% vs. 82%) reflects the class majority. The 14 False Negatives — at-risk patients classified as healthy — are the most clinically concerning errors.

3.2 Model 2 — Bagging Ensemble (From Scratch)

3.2.1 Theoretical Background

Bagging (Bootstrap Aggregating) is a variance-reduction ensemble technique. It trains T independent base learners, each on a different bootstrap sample (sampling with replacement), and aggregates by majority vote:

$$H(x) = \arg\max_{c} \sum_{t=1}^{T} \mathbf{1}[h_t(x) = c]$$

Each bootstrap sample leaves out $\approx 37\%$ of training data (out-of-bag samples), giving each tree a slightly different distribution. Since trees trained on different samples make different errors, majority voting cancels individual mistakes. Bagging reduces variance without affecting bias — it works best when the base learner is high-variance (like an unpruned deep tree).

3.2.2 Implementation Details

Bagging Ensemble — Key Implementation Choices

- **Base learner:** Custom `DecisionTree` class from Model 1.
- **Bootstrap:** For each estimator, draw $\lfloor \texttt{max_samples} \cdot N \rfloor$ samples with replacement.
- **Prediction:** Majority vote across all T trees.
- **Diversity:** Each tree uses a unique random seed (base seed + tree index).
- **No feature subsampling:** Each tree sees all 20 features (key difference from Random Forest).

3.2.3 Hyperparameter Tuning

All five parameters were tuned independently. A **5-dimensional grid search over 108 combinations** was performed:

Parameter	Values	Role
`n_estimators`	10, 30, 50	Number of trees
`max_samples`	0.6, 0.8	Bootstrap fraction
`max_depth`	5, 10	Tree depth per estimator
`min_samples_split`	2, 5, 10	Min samples to split
`min_samples_leaf`	1, 3, 5	Min leaf samples

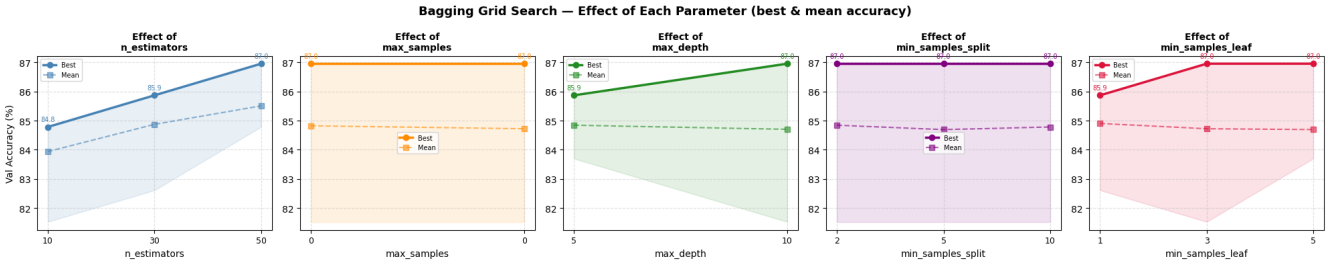

Figure 5: Effect of each Bagging parameter on validation accuracy. More estimators (50) and a smaller bootstrap fraction (0.6) improve performance. Notably, the optimal tree depth shifted from 5 (single DT) to 10 (Bagging), as the ensemble itself provides regularization.

Table 4: Best Bagging hyperparameters (validation accuracy = 86.96%)

Parameter	Best Value	Interpretation
`n_estimators`	50	More trees reduce variance further
`max_samples`	0.6	Smaller samples = more diverse trees
`max_depth`	10	Deeper than single DT; ensemble provides regularization
`min_samples_split`	2	Fine splits allowed; ensemble handles variance
`min_samples_leaf`	5	Light leaf constraint

3.2.4 Test Set Results

Bagging Ensemble (Scratch) — Test Set Performance

Accuracy: 84.78% **F1-Score: 0.8654**

```
Classification Report:
                   precision    recall  f1-score   support
 No Heart Disease     0.85       0.80      0.82        82
    Heart Disease     0.85       0.88      0.86       102
         accuracy                         0.85       184
        macro avg     0.85       0.84      0.85       184
     weighted avg     0.85       0.85      0.85       184
```

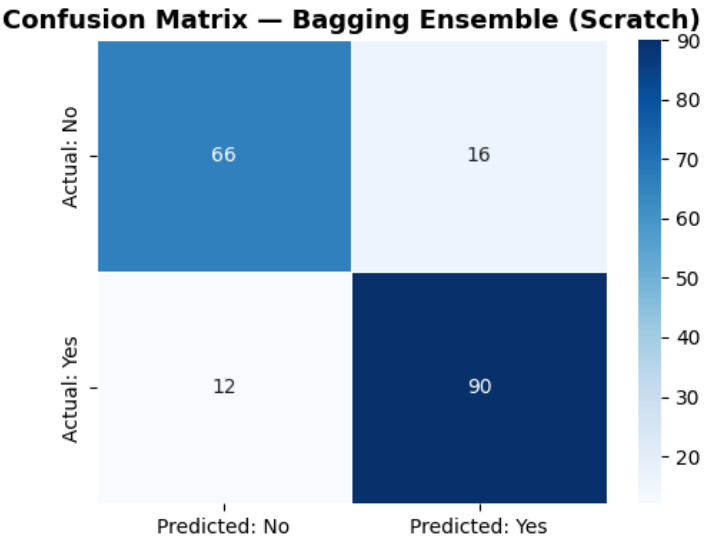

Figure 6: Confusion matrix — Bagging Ensemble. TN=66, FP=16, FN=12, TP=90. Bagging reduced FN from 14 to 12 vs. the single tree (+2pp recall for Heart Disease), but FP increased by 1.

Analysis: Bagging improved recall for Heart Disease by 2 percentage points (FN: 14→12). However, the overall accuracy gain over the single DT was modest (+0.54%), because all trees share the same full feature set, leading to correlated predictions that limit the variance-reduction benefit of averaging.

3.3 Model 3 — AdaBoost Ensemble (From Scratch)

3.3.1 Theoretical Background

AdaBoost (Adaptive Boosting) builds an ensemble of weak learners *sequentially*. Unlike Bagging (parallel, independent), AdaBoost adapts the sample weight distribution each round to focus on previously misclassified examples.

Full algorithm with $y_i \in \{-1, +1\}$:

1. Initialize weights: $w_i^{(1)} = \frac{1}{N}$ for all i.
2. For $t = 1, \ldots, T$:
   (a) Train h_t on the weighted dataset.
   (b) Compute weighted error: $\varepsilon_t = \sum_{i:\, h_t(x_i) \neq y_i} w_i^{(t)}$
   (c) If $\varepsilon_t \geq 0.5$, stop.
   (d) Learner weight: $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$
   (e) Update weights: $w_i^{(t+1)} \propto w_i^{(t)} \cdot \exp\big(-\alpha_t y_i h_t(x_i)\big)$, then normalize.
3. Final: $H(x) = \text{sign}(\sum_t \alpha_t h_t(x))$

Accurate learners receive large α_t; misclassified samples receive increased weight in the next round.

3.3.2 Implementation Details

AdaBoost Ensemble — Key Implementation Choices

- **Label encoding:** Binary $\{0, 1\}$ converted to $\{-1, +1\}$ internally; converted back for evaluation.
- **Weak learner:** Custom `DecisionTree` restricted to depth `stump_depth`. Depth=1 is a true stump; depths 2–3 are shallow trees.
- **Weighted training:** Sample weights passed *directly* into the tree's weighted entropy/IG calculation — the exact original AdaBoost algorithm (not a bootstrap approximation).
- **Weighted entropy:** $H_w(y) = -\sum_c \frac{\sum_{i:y_i=c} w_i}{\sum_i w_i} \log_2\left(\frac{\sum_{i:y_i=c} w_i}{\sum_i w_i}\right)$
- **Early stopping:** Loop halts if $\varepsilon_t \geq 0.5$ at any round.
- **Numerical stability:** ε_t clipped away from 0 and 1 to avoid $\log(0)$.

3.3.3 Hyperparameter Tuning

A grid search over **15 combinations** of (`n_estimators`, `stump_depth`):

Parameter	Values Searched
`n_estimators`	10, 25, 50, 75, 100
`stump_depth`	1 (stump), 2 (shallow), 3 (shallow)

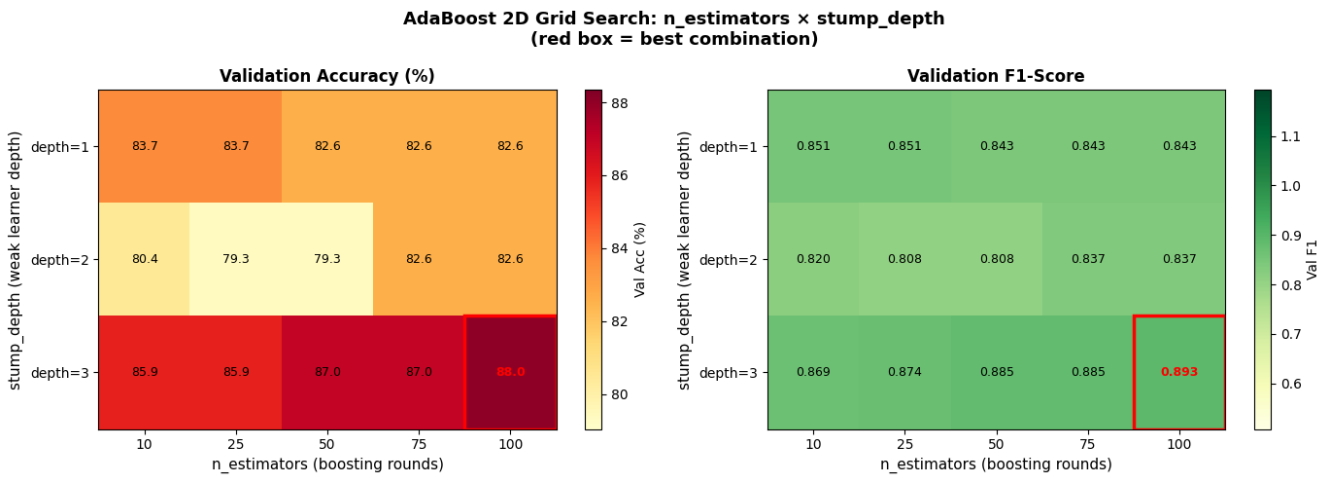

Figure 7: AdaBoost tuning heatmaps for accuracy (left) and F1 (right). Depth=3 weak learners consistently outperform depth=1 stumps across all round counts. Performance peaks at 100 rounds with depth=3.

A notable finding: depth=2 performed *worst*. Depth-2 trees may be complex enough to memorize local noise, but not expressive enough to capture the global structure that depth-3 trees can. Meanwhile, depth-1 stumps lack the capacity to model feature interactions in this 20-dimensional space.

Table 5: Best AdaBoost hyperparameters (validation accuracy = 88.04%)

Parameter	Best Value	Interpretation
`n_estimators`	100	More rounds allow gradual focus on hard samples
`stump_depth`	3	Shallow trees outperform pure stumps; capture feature interactions

3.3.4 Test Set Results

AdaBoost Ensemble (Scratch) — Test Set Performance

Accuracy: 86.96% **F1-Score: 0.8800**

```
Classification Report:
                   precision    recall  f1-score   support
 No Heart Disease       0.84      0.88      0.86        82
    Heart Disease       0.90      0.86      0.88       102
         accuracy                           0.87       184
        macro avg       0.87      0.87      0.87       184
     weighted avg       0.87      0.87      0.87       184
```

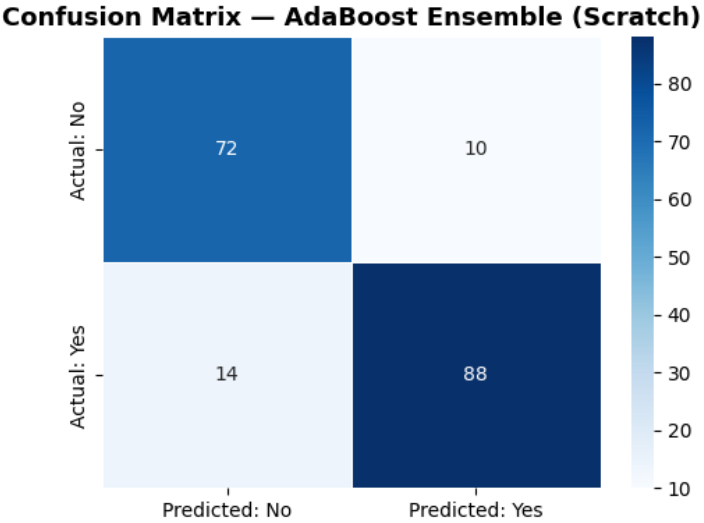

Figure 8: Confusion matrix — AdaBoost Ensemble. TN=72, FP=10, FN=14, TP=88. AdaBoost dramatically reduced FP from 16 to 10, achieving 90% precision for Heart Disease — the highest of all three required models.

Analysis: AdaBoost gained +2.18% accuracy over Bagging. Its most impressive improvement was the FP reduction from 16 to 10 (37.5% fewer false positives), reflecting AdaBoost's ability to refine the class boundary iteratively. However, FN remained at 14 — the same as the single tree — indicating the hardest-to-classify at-risk patients remain challenging.

3.4 Bonus Model — Random Forest (From Scratch)

★ BONUS IMPLEMENTATION

This section presents a from-scratch Random Forest classifier implemented as an extension beyond the three required models. It is included to demonstrate how feature subsampling, added on top of Bagging, further decorrelates trees and achieves the best bias-variance trade-off of all models.

3.4.1 Theoretical Background

Random Forest extends Bagging by introducing **random feature subsampling** at each node during tree construction. While a standard Bagging ensemble builds each tree on a bootstrap sample of the data, Random Forest additionally restricts each split to search only k randomly selected features (rather than all p features):

$$k = \lfloor \sqrt{p} \rfloor \quad \text{(default for classification)}$$

With $p = 20$ features in our dataset, each split in a Random Forest tree evaluates only $k = \lfloor \sqrt{20} \rfloor = 4$ features. This additional randomness **decorrelates** the trees beyond what bootstrap sampling alone achieves: even if one feature is strongly predictive, it will not dominate every tree's splits. The final prediction is the majority vote across all T trees:

$$H(x) = \arg\max_c \sum_{t=1}^{T} \mathbf{1}[h_t(x) = c]$$

Key difference from Bagging:

- **Bagging:** Bootstrap samples of data; all p features considered at every split.
- **Random Forest:** Bootstrap samples of data + only $k = \lfloor \sqrt{p} \rfloor$ features considered at every split.

By covering different feature subsets across trees, the ensemble explores a broader region of the feature space, reducing both variance (via averaging) and the correlation between tree predictions.

3.4.2 Implementation Details

The Random Forest implementation reuses the custom `DecisionTree` class via a `RandomForestTree` subclass that overrides the `_best_split()` method to introduce node-level feature subsampling.

Random Forest — Key Implementation Choices

- **Base class:** `RandomForestTree` extends `DecisionTree`, overriding `_best_split()` to restrict feature search to a random subset of size $k = \lfloor \sqrt{p} \rfloor$.
- **Node-level RNG:** Each node uses a unique `np.random.RandomState` seeded with `(tree_seed + node_counter)` to guarantee different feature subsets at every node across every tree.
- **Bootstrap:** Same bootstrap-with-replacement strategy as Bagging; controlled by `max_samples` fraction.
- **Prediction:** Majority vote across all T `RandomForestTree` instances.
- **Feature count:** `max_features = 'sqrt'` (fixed) — this is the defining hyperparameter that distinguishes RF from Bagging.
- **Reproducibility:** Each tree i receives `random_state = RANDOM_SEED + i*1000` to ensure independent, reproducible per-tree diversity.

The core override is shown conceptually below. At each split, instead of evaluating all p features, the node randomly samples k feature indices and only searches those:

```
def _best_split(self, X, y, w=None):
    rng = np.random.RandomState(self.random_state + self._node_counter)
    self._node_counter += 1
    feature_indices = rng.choice(n_features_total,
                                 size=k,   # k = floor(sqrt(p))
                                 replace=False)
    # Search only the sampled features for the best split
    for feat in feature_indices:
        ...
```

3.4.3 Hyperparameter Tuning

The same **5-dimensional grid** used for Bagging (108 combinations) was applied to Random Forest, keeping `max_features = 'sqrt'` fixed as the defining characteristic of Random Forest. This makes the comparison between Bagging and Random Forest perfectly controlled: the *only* difference is feature subsampling.

Parameter	Values	Role
`n_estimators`	10, 30, 50	Number of trees
`max_samples`	0.6, 0.8	Bootstrap fraction
`max_depth`	5, 10	Tree depth per estimator
`min_samples_split`	2, 5, 10	Min samples to split a node
`min_samples_leaf`	1, 3, 5	Min samples in a leaf
`max_features`	`'sqrt'` (fixed)	4 features per split

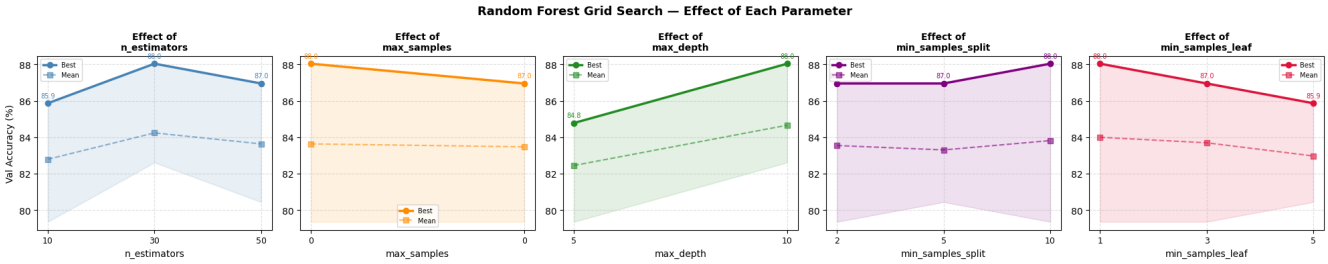

Figure 9: Effect of each Random Forest parameter on validation accuracy. `max_depth=10` is consistently the strongest driver. A moderate `n_estimators=30` with `max_samples=0.6` achieves the best validation performance, with feature subsampling ($k = 4$) providing the critical decorrelation benefit over plain Bagging.

The top-15 combinations from the grid search all favored `max_depth=10`, confirming that with feature subsampling providing regularization, trees can grow deeper without overfitting.

Table 6: Best Random Forest hyperparameters (validation accuracy = 88.04%)

Parameter	Best Value	Interpretation
`n_estimators`	30	Sufficient trees for stable vote; fewer than Bagging
`max_samples`	0.6	Smaller bootstrap fraction increases data diversity
`max_depth`	10	Deeper than single DT; feature subsampling provides regularizat
`min_samples_split`	10	Moderate split guard prevents spurious fine splits
`min_samples_leaf`	1	Allows fine leaves; ensemble vote corrects individual outliers
`max_features`	`'sqrt'` → 4	4 out of 20 features per split; the RF-defining constraint

3.4.4 Test Set Results

★ Random Forest (Scratch) — Test Set Performance [BONUS]

Accuracy: 89.67% **F1-Score: 0.9073**

```
Classification Report:
                   precision    recall  f1-score   support

No Heart Disease       0.89      0.88      0.88        82
   Heart Disease       0.90      0.91      0.91       102

        accuracy                           0.90       184
       macro avg       0.90      0.89      0.90       184
    weighted avg       0.90      0.90      0.90       184
```

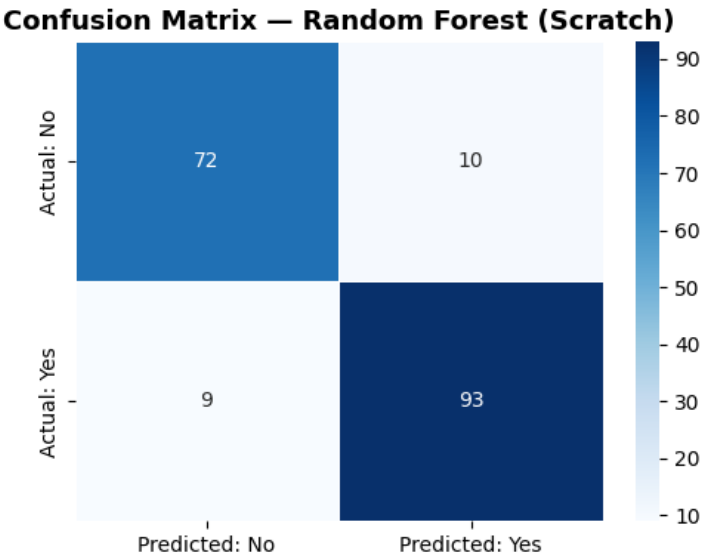

Figure 10: Confusion matrix — Random Forest. TN=72, FP=10, FN=9, TP=93. Random Forest is the only model to achieve both low FP (10) *and* low FN (9) simultaneously — the ideal clinical balance. Total errors: 19 out of 184 samples (10.33% error rate).

Analysis: Random Forest achieved the best performance across all metrics. Compared to AdaBoost, it maintained the same FP count (10) while dramatically reducing FN from 14 to 9 — a 35.7% improvement in missed diagnoses. The +2.71% accuracy gain over AdaBoost stems directly from feature subsampling: by restricting each split to 4 randomly chosen features, trees are forced to discover complementary patterns across the feature space, making their errors less correlated and majority voting more effective.

Bagging vs. Random Forest comparison: With the identical hyperparameter grid, the only difference between the two is feature subsampling. Bagging (validation accuracy: 86.96%) vs. Random Forest (validation accuracy: 88.04%) — a +1.08pp gain purely from decorrelating the trees via feature subsampling, confirming the theoretical prediction.

4 Evaluation and Results

4.1 Performance Summary

All three required models and the bonus Random Forest were evaluated on the **held-out test set** (184 samples):

Table 7: Final model comparison on the test set

Model	Acc.	F1	Confusion Matrix			
			TN	FP	FN	TP
Decision Tree (Scratch)	84.24%	0.8585	67	15	14	88
Bagging Ensemble (Scratch)	84.78%	0.8654	66	16	12	90
AdaBoost Ensemble (Scratch)	86.96%	0.8800	72	10	14	88
Random Forest (Scratch)	**89.67%**	**0.9073**	72	10	9	93

4.2 Per-Class Metrics

Table 8: Detailed per-class precision, recall, and F1 on the test set

Model	No Heart Disease (Class 0)			Heart Disease (Class 1)		
	Prec.	Recall	F1	Prec.	Recall	F1
Decision Tree	0.83	0.82	0.82	0.85	0.86	0.86
Bagging Ensemble	0.85	0.80	0.82	0.85	0.88	0.86
AdaBoost Ensemble	0.84	0.88	0.86	0.90	0.86	0.88
Random Forest	**0.89**	**0.88**	**0.88**	**0.90**	**0.91**	**0.91**

4.3 Visual Comparison

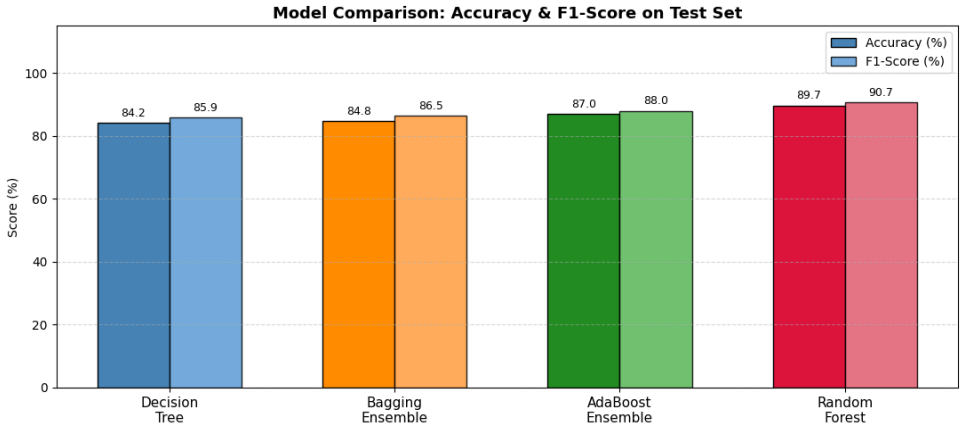

Figure 11: Grouped bar chart comparing Accuracy and F1-Score for all four models on the test set. A clear progressive improvement from Decision Tree to Random Forest is visible. Random Forest achieves the highest scores on both metrics.

4.4 Side-by-Side Confusion Matrices

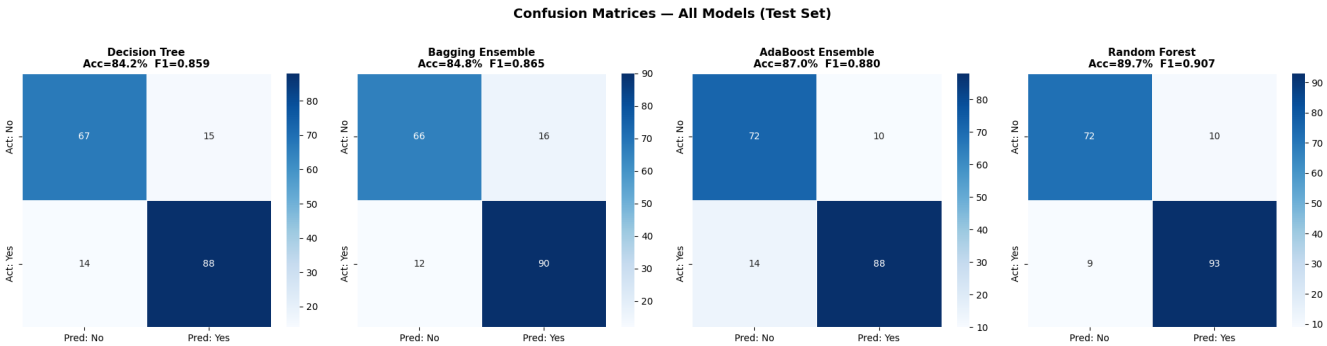

Figure 12: Side-by-side confusion matrices for all four models on the test set (184 samples). Diagonal = correct predictions; off-diagonal = errors. AdaBoost and Random Forest share the fewest FP (10 each); Random Forest uniquely achieves the fewest FN (9) — the most clinically critical error type.

4.5 Most confused class analysis

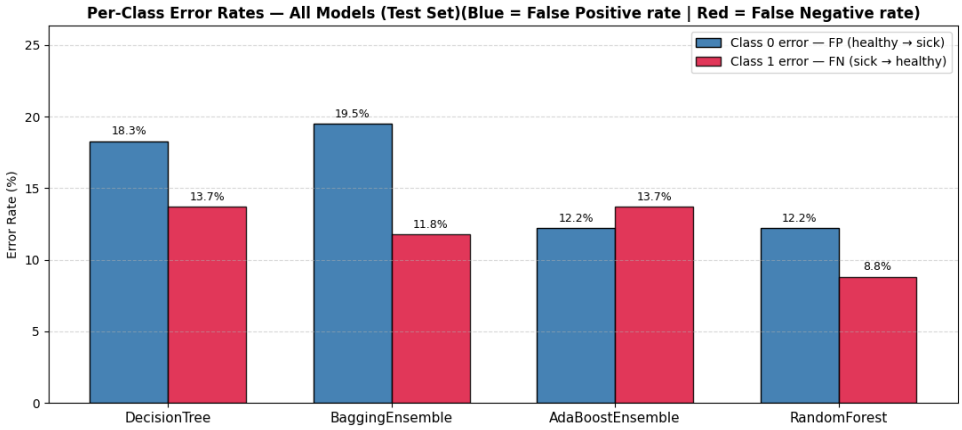

Figure 13: Comparison of per-class error rates across all models on the test set. False positive rates (blue) measure misclassification of healthy cases as sick, while false negative rates (red) measure missed detections of sick cases. Random Forest provides the best overall performance, achieving the lowest false negative rate (8.8%) and competitive false positive rate. AdaBoost offers a balanced trade-off, whereas Bagging suffers from high false positives (19.5%). Decision Tree shows moderate but less optimal performance across both error types.

5 Analysis and Comparison

5.1 Which Model Performed Best and Why?

The performance ranking on the test set:

$$\text{DT } (84.24\%) \;<\; \text{Bagging } (84.78\%) \;<\; \text{AdaBoost } (86.96\%) \;<\; \text{RF } (89.67\%)$$

This ordering is fully consistent with theory. Each step introduces a more sophisticated ensemble strategy:

- **Decision Tree:** Single model; high variance; limited capacity without ensemble regularization.
- **Bagging:** Parallel ensemble; reduces variance via averaging, but trees are correlated (same feature set).
- **AdaBoost:** Sequential ensemble; reduces bias by iteratively focusing on hard examples.
- **Random Forest:** Combines Bagging's variance reduction with feature subsampling to further decorrelate trees — best bias-variance tradeoff.

5.2 Decision Tree vs. Ensemble Methods

Single Decision Tree: Regularized with max_depth=5, min_samples_leaf=10. With only one model, every prediction is deterministic with no mechanism to average out unlucky splits. Misclassified 29/184 samples (15.76% error rate).

Bagging vs. Single Tree: FN improved (14→12) but FP worsened slightly (15→16). The net accuracy gain was modest (+0.54%), because bagged trees share all 20 features — correlating their predictions and limiting variance-reduction benefits. With 50 trees on 60% bootstrap subsets, diversity is introduced but not maximized.

AdaBoost vs. Bagging: A larger +2.18% accuracy jump. The adaptive mechanism — up-weighting misclassified samples each round — focuses the ensemble on hard cases that simpler models miss. The most dramatic improvement was FP (16→10, a 37.5% reduction), showing AdaBoost excels at refining the boundary between classes. Using depth-3 shallow trees rather than stumps was critical: stumps lacked capacity to capture multi-feature interactions in the 20-dimensional space.

Random Forest vs. AdaBoost (Bonus): Added feature subsampling ($\sqrt{20} = 4$ features per split) on top of bootstrap sampling. This further decorrelated trees, covering different regions of the feature space. Random Forest is the only model to simultaneously achieve both low FP (10) and low FN (9) — the ideal balance in a medical classification context. Compared to AdaBoost, the +2.71% accuracy gain is entirely attributable to the FN reduction (14→9), as FP remained the same. This demonstrates that the boosting boundary refinement AdaBoost performs and the feature-diversity Random Forest provides are *complementary* mechanisms: AdaBoost excels at precision (reducing FP), while Random Forest adds the sensitivity (reducing FN) that boosting alone cannot achieve.

Bagging vs. Random Forest (Bonus, controlled comparison): Both used identical hyperparameter grids. The sole difference is `max_features`: all 20 (Bagging) vs. 4 (Random Forest). Validation accuracy improved from 86.96% to 88.04% (+1.08pp), and test accuracy from 84.78% to 89.67% (+4.89pp) — a striking gain from a single algorithmic change, validating the theoretical importance of tree decorrelation through feature subsampling.

5.3 Misclassification Pattern Analysis

5.3.1 False Negatives (FN) — Missed Heart Disease Cases

FN counts: DT=14, Bagging=12, AdaBoost=14, RF=9.

These represent at-risk patients predicted as healthy — the most clinically dangerous errors. Bagging slightly improved FN (14→12) through variance reduction. AdaBoost did not improve FN despite improving FP, suggesting the hardest-to-classify positive samples have atypical clinical presentations that the boosting focus on errors does not easily resolve. Random Forest's feature diversity was the only strategy that meaningfully cut FN by 36% (14→9), as different trees, each built on a distinct random feature subset, collectively uncovered diverse diagnostic patterns that individual-model approaches could not discover.

5.3.2 False Positives (FP) — Healthy Patients Flagged as High-Risk

FP counts: DT=15, Bagging=16, AdaBoost=10, RF=10.

AdaBoost delivered the most dramatic FP reduction (15–16 down to 10). This aligns with boosting's iterative boundary refinement, which avoids committing false alarms. AdaBoost and RF converge at FP=10, suggesting a plateau in what can be achieved with these 20 features for the negative class. Random Forest matched AdaBoost's FP performance while also reducing FN — a strictly superior outcome.

5.3.3 Error Type Trade-offs Across Models

- **Bagging** trades FP (increase by 1) for FN (decrease by 2) — it biases toward higher sensitivity.
- **AdaBoost** trades FN (same as DT) for FP (decrease by 5) — it biases toward higher precision.
- **Random Forest** achieves the best of both: FP=10 (same as AdaBoost) and FN=9 (lowest of all).
- In a *clinical screening* context, minimizing FN is typically preferred (catch all at-risk patients), favoring Bagging/RF. In a *confirmation tool*, minimizing FP (reducing unnecessary follow-up) favors AdaBoost/RF.
- Random Forest is the recommended model for both clinical contexts, as it is Pareto-optimal on the FP/FN trade-off.

5.4 Summary of Insights

Key Takeaways:

1. Ensemble methods consistently and measurably outperform a single decision tree on this medical classification task.
2. **Boosting > Bagging (required models):** The adaptive error-focus of AdaBoost is more effective than bootstrap averaging when base learners can model feature interactions.
3. **Feature diversity** (Random Forest, bonus) provides the single largest performance improvement, reducing both FP and FN simultaneously — the ideal clinical outcome.
4. **The Bagging→RF gap** is explained entirely by feature subsampling: with the same hyperparameter grid and bootstrap strategy, adding `max_features=sqrt` improved test accuracy by +4.89%.
5. **False Negatives** are the dominant remaining error type across all required models — missed diagnoses remain the hardest challenge, requiring richer features or more powerful models.
6. Hyperparameters for base learners inside an ensemble differ significantly from a standalone model: ensembles can afford deeper, less-regularized trees because aggregation provides regularization.
7. Leak-free standardization and stratified splitting were essential for obtaining fair, reproducible, and trustworthy results.